Procesamiento Funcional de Datos para un Sistema de Gestión de Transporte Urbano: Módulo TecnoMovil Data en Java

Evidencia de Aprendizaje 2: Creación de Repositorio

Pablo José Ramírez Acevedo

Grupo 13 (trabajo individual)

Ingeniería de Software y Ciencia de Datos, Institución Universitaria Digital de Antioquia

Programación Orientada a Objetos

Docente: Leonardo Montes Gutierrez

30 de octubre de 2026

Introducción

La ciudad de TecnoValle moviliza cada día a más de 450.000 pasajeros en buses, rutas alimentadoras y estaciones principales, y su sistema de información recibe entre 6 y 12 millones de registros diarios provenientes de validadores de tarjetas, sensores de estación, aplicaciones móviles y dispositivos IoT instalados en los buses (Prada Bohórquez, 2026). La plataforma actual, construida con un enfoque imperativo y altamente acoplado, procesa estos datos de forma secuencial sobre estructuras compartidas y mutables. El resultado son jornadas de procesamiento de hasta dos horas, errores en los conteos por condiciones de carrera y un código extenso y costoso de mantener.

Para responder a esta situación, la Alcaldía creó el proyecto TecnoMovil Data, y sus ingenieros identificaron como módulo más crítico el de procesamiento y agregación de los datos diarios. La programación funcional ofrece una solución directa a los problemas descritos: al trabajar con funciones puras y datos inmutables desaparecen los efectos secundarios que originan las condiciones de carrera, y al expresar el procesamiento como una secuencia declarativa de transformaciones con la API de Streams de Java, el mismo código puede ejecutarse en paralelo sin cambiar su lógica (Urma et al., 2018).

Este informe presenta el desarrollo de la Evidencia de Aprendizaje 2, realizada de forma individual. Se describe el diseño y la implementación en Java 17, sin frameworks externos, de un módulo que resuelve las seis tareas del caso de estudio: afluencia por estación, horas pico, rutas más utilizadas, patrones de viaje por usuario, tiempo promedio entre estaciones y detección de sobrecarga. Además, se documentan las funciones puras, la inmutabilidad, las funciones de orden superior, las lambdas y los streams utilizados; se presentan las pruebas y la evidencia de

ejecución sobre más de dos millones de registros simulados, y se indica el repositorio de GitHub donde se aloja la solución. El código fuente completo se incluye en el Apéndice A.

Objetivos

Objetivo General

Diseñar e implementar en Java, sin frameworks externos, un módulo funcional de procesamiento y agregación de datos para el sistema de transporte urbano de TecnoValle, aplicando funciones puras, inmutabilidad, funciones de orden superior, lambdas y streams, y alojar la solución en la rama principal de un repositorio de GitHub.

Objetivos Específicos

1. Implementar con funciones puras y pipelines de Streams las seis operaciones requeridas: afluencia por estación, horas pico, rutas más utilizadas, patrones de viaje, tiempo promedio entre estaciones y detección de sobrecarga.
2. Procesar de manera declarativa un volumen de más de dos millones de registros simulados sin modificar las colecciones originales.
3. Verificar, con pruebas que no dependen de frameworks externos, la exactitud de los resultados y que la ejecución secuencial y la paralela (`parallelStream()`) producen resultados idénticos.
4. Crear un repositorio en GitHub y publicar en la rama principal (`main`) el código fuente, la documentación y el informe del trabajo.

Desarrollo

Análisis del Caso de Estudio

El caso de estudio pide diseñar un módulo funcional en Java, basado en Streams, lambdas, funciones puras y colecciones inmutables, capaz de procesar un conjunto grande de

datos simulados. Cada registro contiene los cinco campos de la Tabla 1. El enunciado impone además seis consideraciones técnicas: usar funciones puras, no modificar las listas originales, emplear lambdas y pipelines con Streams, tratar las colecciones de forma declarativa, permitir la paralelización con `parallelStream()` y evitar por completo los efectos secundarios (Prada Bohórquez, 2026). Estas restricciones guiaron todas las decisiones de diseño que se describen a continuación.

Tabla 1

Estructura del Registro Procesado

Campo	Tipo en Java	Descripción
`idUsuario`	`String`	Identificador del usuario (por ejemplo, U0000001)
`ruta`	`String`	Código de la ruta: T1, T2, C3, A1, A2 o E1
`estacion`	`String`	Estación en la que se realiza la validación
`accion`	`Accion` (enum)	`ENTRADA` al ingresar a la estación; `SALIDA` al salir o cambiar de ruta
`timestamp`	`LocalDateTime`	Fecha y hora de la validación

Nota. Elaboración propia a partir de Prada Bohórquez (2026).

Diseño de la Solución

La solución separa un **núcleo funcional** de una **capa imperativa**. Todo el procesamiento se concentra en funciones puras, y los efectos inevitables, como leer un archivo, medir tiempos e imprimir en consola, quedan aislados en las clases de los bordes. El proyecto se organizó en cuatro paquetes y un paquete de pruebas (Tabla 2). Solo se usó el JDK 17, cuyos records y cuya API de Streams son suficientes para el problema (Oracle, 2021a, 2021b).

Tabla 2

Organización del Proyecto TecnoMovilData

Paquete	Clases	Responsabilidad
`tecnomovil.modelo`	Registro, Accion, Tramo, EstadisticaTiempos, EstadoRuta	Tipos de datos inmutables (records y enum)
`tecnomovil.datos`	GeneradorDatos, LectorCsv	Obtención de registros: generador determinista y lectura de archivos CSV
`tecnomovil.analisis`	AnalizadorTransporte, ResultadoAnalisis	Núcleo funcional: funciones puras de las tareas a–f
`tecnomovil.app`	Main	Capa imperativa: carga de datos, medición de tiempos e impresión
`tecnomovil.pruebas`	PruebasAnalizador	21 pruebas automatizadas sin frameworks externos

Nota. Elaboración propia.

Cada función del analizador recibe la lista de registros y un parámetro booleano `paralelo`, con el que decide si crea el flujo con `stream()` o con `parallelStream()`. Como ninguna función altera la lista ni comparte estado, cambiar el modo de ejecución no exige modificar la lógica. Los resultados de las seis tareas se agrupan en el record `ResultadoAnalisis`, cuya igualdad estructural permite comparar en una sola instrucción la ejecución secuencial con la paralela.

Generación de los Datos Simulados

Para trabajar con un volumen comparable al del caso, la clase `GeneradorDatos` produce los registros de un día para 450.000 usuarios, la cifra de pasajeros diarios de TecnoValle. La red simulada tiene seis rutas (dos troncales, una circular, dos alimentadoras y una expresa al aeropuerto) y 21 estaciones. Cada usuario hace entre uno y cuatro viajes; el segundo viaje es el regreso del primero, lo que reproduce el patrón casa-trabajo-casa. La demanda por hora tiene picos en la mañana y en la tarde, y en las horas pico los recorridos tardan un 20 % más. Cada

viaje produce un registro de entrada y uno de salida. En total se generaron 2.023.878 registros, y el programa acepta como parámetro un número mayor de usuarios para alcanzar los 6 a 12 millones de registros descritos en el caso.

El generador no usa `java.util.Random`, porque su estado interno cambia en cada llamada y, compartido entre hilos, introduciría efectos secundarios. En su lugar, cada valor pseudoaleatorio se calcula con una función hash pura a partir de la semilla, el usuario, el viaje y el campo, usando la función de mezcla del algoritmo SplitMix64 (Steele et al., 2014). La misma entrada produce siempre el mismo valor, de modo que los datos son reproducibles y pueden generarse con un `IntStream` paralelo sin condiciones de carrera. La lista final se ordena por fecha y hora, como llegarían los eventos desde los validadores, y se materializa con `toList()`, que devuelve una lista no modificable.

```
public static List<Registro> generar(int usuarios, long semilla, LocalDate fecha) {
    return IntStream.range(0, usuarios)
            .parallel()
            .boxed()
            .flatMap(u -> registrosDeUsuario(u, semilla, fecha))
            .sorted(Comparator.comparing(Registro::timestamp)
                    .thenComparing(Registro::idUsuario))
            .toList();
}

static long hash(long semilla, long... claves) {
    return LongStream.of(claves)
            .reduce(mezclar(semilla), (acum, clave) -> mezclar(acum ^ mezclar(clave)));
}

static long mezclar(long x) {
    long z = x + 0x9E3779B97F4A7C15L;
    z = (z ^ (z >>> 30)) * 0xBF58476D1CE4E5B9L;
    z = (z ^ (z >>> 27)) * 0x94D049BB133111EBL;
    return z ^ (z >>> 31);
}
```

Implementación de las Tareas Requeridas

La Tabla 3 relaciona cada requerimiento con los métodos de la clase `AnalizadorTransporte` que lo resuelven y con las operaciones de Stream que utiliza. En los apartados siguientes se explica cada uno y se muestra su código.

Tabla 3

Correspondencia entre Requerimientos y Operaciones Funcionales

Requerimiento	Métodos	Operaciones principales
a) Afluencia por estación	`afluenciaPorEstacion`, `usuariosUnicosPorEstacion`	`filter`, `groupingBy`, `counting`, `mapping`, `toSet`
b) Horas pico	`flujoPorHora`, `horasPico`	`groupingBy` con `TreeMap`, `counting`, `sorted`, `limit`
c) Rutas más utilizadas	`usoPorRuta`, `rutasMasUtilizadas`	`filter`, `groupingBy`, `counting`, `sorted`, `limit`
d) Patrones de viaje	`patronesDeViaje`	`groupingBy`, `mapping`, `toUnmodifiableList`
e) Tiempo entre estaciones	`tramos`, `tiempoEntreEstaciones`	`groupingBy`, `flatMap`, `IntStream.range`, `averagingLong`
f) Sobrecarga en rutas	`detectarSobrecarga`, `evaluarRuta`	`groupingBy` anidado, `max`, `map`, `sorted`

Nota. Elaboración propia.

a) Cálculo de Afluencia por Estación

La afluencia se calcula con los registros de entrada, pues cada uno representa a un usuario que ingresa a una estación. Para no repetir lógica se construyó la función de orden superior `contarPor`, que recibe un predicado (qué registros contar) y una función clasificadora (por qué clave agrupar). Así, la afluencia es simplemente `contarPor` con el predicado `ES_ENTRADA` y la referencia a método `Registro::estacion`. Como un mismo usuario puede ingresar varias veces a una estación en el día, se añadió `usuariosUnicosPorEstacion`, que reúne los identificadores en un conjunto y toma su tamaño.

```
public static <K> Map<K, Long> contarPor(List<Registro> registros,
                                         Predicate<? super Registro> filtro,
                                         Function<? super Registro, ? extends K> clave,
                                         boolean paralelo) {
    return flujo(registros, paralelo)
            .filter(filtro)
            .collect(collectingAndThen(groupingBy(clave, counting()),
                    Collections::unmodifiableMap));
}

public static Map<String, Long> afluenciaPorEstacion(List<Registro> registros,
                                                     boolean paralelo) {
    return contarPor(registros, ES_ENTRADA, Registro::estacion, paralelo);
}

public static Map<String, Integer> usuariosUnicosPorEstacion(List<Registro> registros,
                                                             boolean paralelo) {
    return flujo(registros, paralelo)
            .filter(ES_ENTRADA)
            .collect(collectingAndThen(
                    groupingBy(Registro::estacion,
                            collectingAndThen(mapping(Registro::idUsuario, toSet()),
                                    Set::size)),
                    Collections::unmodifiableMap));
}
```

Estación Central fue la estación de mayor afluencia, con 143.150 ingresos de 135.243 usuarios distintos, casi el doble que la segunda (Parque Bolívar, con 74.267). Este resultado es coherente con la red, porque Estación Central es el punto de conexión de cuatro de las seis rutas.

b) Identificación de Horas Pico

Los registros se agrupan por hora del día con la función `HORA_DEL_DIA`, obtenida por composición: `Registro::timestamp` seguida de `LocalDateTime::getHour` mediante `andThen`. Se cuentan todos los registros, entradas y salidas, porque ambos representan flujo en las estaciones. El agrupamiento usa un `TreeMap` para que el resultado quede ordenado por hora, y la función genérica `topN` ordena de forma descendente y conserva las tres horas con más registros. En caso de empate, `topN` desempata por la clave para que el resultado sea determinista.

```
public static final Function<Registro, Integer> HORA_DEL_DIA =
        ((Function<Registro, LocalDateTime>) Registro::timestamp)
                .andThen(LocalDateTime::getHour);

public static SortedMap<Integer, Long> flujoPorHora(List<Registro> registros,
                                                    boolean paralelo) {
    return flujo(registros, paralelo)
            .collect(collectingAndThen(
                    groupingBy(HORA_DEL_DIA, TreeMap::new, counting()),
                    Collections::unmodifiableSortedMap));
}

public static List<Map.Entry<Integer, Long>> horasPico(List<Registro> registros, int n,
                                                       boolean paralelo) {
    return topN(flujoPorHora(registros, paralelo), n);
}
```

Las horas pico fueron las 18:00 (233.328 registros), las 07:00 (231.884) y las 17:00 (215.218). La hora valle fue las 22:00, con 21.418 registros, casi once veces menos que la hora de mayor flujo.

c) Rutas Más Utilizadas

El volumen de uso de una ruta se mide como el número de viajes que se inician en ella, es decir, sus registros de entrada. Se reutiliza `contarPor`, ahora con `Registro::ruta` como clasificador, y `topN` produce el ranking. Este caso muestra la ventaja de las funciones de orden superior: dos requerimientos distintos se resuelven con la misma función cambiando solo la clave de agrupación.

```
public static Map<String, Long> usoPorRuta(List<Registro> registros, boolean paralelo) {
    return contarPor(registros, ES_ENTRADA, Registro::ruta, paralelo);
}

public static List<Map.Entry<String, Long>> rutasMasUtilizadas(List<Registro> registros,
                                                               int n, boolean paralelo) {
    return topN(usoPorRuta(registros, paralelo), n);
}

public static <K extends Comparable<? super K>, V extends Comparable<? super V>>
        List<Map.Entry<K, V>> topN(Map<K, V> valores, int n) {
    return valores.entrySet().stream()
            .sorted(Map.Entry.<K, V>comparingByValue().reversed()
                    .thenComparing(Map.Entry.<K, V>comparingByKey()))
            .limit(n)
            .map(e -> Map.entry(e.getKey(), e.getValue()))
            .toList();
}
```

La troncal T1 fue la ruta más utilizada, con 304.401 viajes (30,1 %), seguida de la troncal T2, con 252.745 (25,0 %), y de la circular C3, con 151.686 (15,0 %).

d) Patrones de Viaje por Usuario

Para cada usuario se construye la lista de estaciones visitadas en el orden en que aparecen en los registros. Se agrupa por `idUsuario` y, con el colector `mapping`, cada registro se transforma en su estación y se acumula en una lista no modificable. El colector `groupingBy` conserva el orden de encuentro dentro de cada grupo incluso cuando el flujo es paralelo, por lo que el orden de las estaciones no se altera (Oracle, 2021a).

```
public static Map<String, List<String>> patronesDeViaje(List<Registro> registros,
                                                         boolean paralelo) {
    return flujo(registros, paralelo)
            .collect(collectingAndThen(
                    groupingBy(Registro::idUsuario, TreeMap::new,
                            mapping(Registro::estacion, toUnmodifiableList())),
                    Collections::unmodifiableSortedMap));
}
```

Se obtuvieron 450.000 patrones, uno por usuario. Por ejemplo, el usuario U0000002 registra *Mercado → Portal Oriente → Portal Oriente → Mercado*: un viaje de ida y su regreso. La estación intermedia aparece dos veces porque en ella el usuario sale y, más tarde, vuelve a entrar.

e) Cálculo de Tiempo Promedio entre Estaciones

Se definió como **tramo** el desplazamiento de un usuario entre dos estaciones consecutivas: una entrada en la estación A seguida inmediatamente de una salida en otra estación B. Para encontrarlos, los registros se agrupan por usuario; los eventos de cada usuario se ordenan por fecha y hora en una lista nueva y se recorren por pares consecutivos con `IntStream.range(0, n - 1)`. La función pura `Tramo.entre` devuelve un `Optional` con el tramo solo si el par es un viaje, lo que descarta los trasbordos (salida y entrada en la misma estación) y el tiempo que pasa entre una salida y la siguiente entrada. Con `flatMap(Optional::stream)` se conservan únicamente los tramos válidos. Por último, `averagingLong` calcula el promedio de cada tramo y `summaryStatistics` el promedio

general. Los tiempos se manejan en segundos enteros (`long`), cuya suma es exacta, para que el resultado secuencial y el paralelo coincidan exactamente.

```
public static Optional<Tramo> entre(Registro a, Registro b) {
    boolean esViaje = a.esEntrada() && b.esSalida() && !a.estacion().equals(b.estacion());
    return esViaje
            ? Optional.of(new Tramo(a.estacion(), b.estacion(),
                    Duration.between(a.timestamp(), b.timestamp()).toSeconds()))
            : Optional.empty();
}

public static List<Tramo> tramos(List<Registro> registros, boolean paralelo) {
    Map<String, List<Registro>> porUsuario = flujo(registros, paralelo)
            .collect(groupingBy(Registro::idUsuario));
    return flujo(porUsuario.values(), paralelo)
            .flatMap(AnalizadorTransporte::tramosDeUsuario)
            .toList();
}

static Stream<Tramo> tramosDeUsuario(List<Registro> eventos) {
    List<Registro> ordenados = eventos.stream()
            .sorted(Comparator.comparing(Registro::timestamp))
            .toList();
    return IntStream.range(0, ordenados.size() - 1)
            .mapToObj(i -> Tramo.entre(ordenados.get(i), ordenados.get(i + 1)))
            .flatMap(Optional::stream);
}

public static EstadisticaTiempos tiempoEntreEstaciones(List<Registro> registros,
                                                       boolean paralelo) {
    List<Tramo> tramos = tramos(registros, paralelo);
    LongSummaryStatistics general = flujo(tramos, paralelo)
            .mapToLong(Tramo::segundos)
            .summaryStatistics();
    Map<String, Double> porTramo = flujo(tramos, paralelo)
            .collect(groupingBy(Tramo::nombre,
                    collectingAndThen(averagingLong(Tramo::segundos), seg -> seg / 60.0)));
    return new EstadisticaTiempos(general.getCount(), general.getAverage() / 60.0, porTramo);
}
```

Se analizaron 1.011.939 tramos, con un tiempo promedio general de 9,8 minutos. Los tramos más largos fueron los que recorren toda la troncal T1, entre Portal Sur y Portal Norte, con 23,3 minutos en promedio.

f) Detección de Sobrecarga en Rutas

La ocupación de una ruta se estima como la relación entre los ingresos de su hora pico y su capacidad horaria simulada, que resulta de multiplicar los buses por hora por la capacidad de cada bus. `detectarSobrecarga` es una función de orden superior: la capacidad no está escrita dentro del análisis, sino que se recibe como un parámetro de tipo `Function<String, Integer>`, y el umbral como un `double`. Así, el mismo código sirve para cualquier escenario de

capacidades. Toda ruta cuya ocupación supera el umbral de 90 % se marca como crítica. La función pura `evaluarRuta` calcula la hora pico, la ocupación y el estado de cada ruta.

```
public static List<EstadoRuta> detectarSobrecarga(List<Registro> registros,
                                                   Function<String, Integer> capacidadHoraria,
                                                   double umbral,
                                                   boolean paralelo) {
    Map<String, Map<Integer, Long>> ingresosPorRutaYHora = flujo(registros, paralelo)
            .filter(ES_ENTRADA)
            .collect(groupingBy(Registro::ruta, groupingBy(HORA_DEL_DIA, counting())));
    return ingresosPorRutaYHora.entrySet().stream()
            .map(e -> evaluarRuta(e.getKey(), e.getValue(),
                    capacidadHoraria.apply(e.getKey()), umbral))
            .sorted(Comparator.comparingDouble(EstadoRuta::ocupacion).reversed()
                    .thenComparing(EstadoRuta::ruta))
            .toList();
}

static EstadoRuta evaluarRuta(String ruta, Map<Integer, Long> ingresosPorHora,
                              int capacidad, double umbral) {
    Comparator<Map.Entry<Integer, Long>> porIngresos = Map.Entry.comparingByValue();
    Map.Entry<Integer, Long> pico = ingresosPorHora.entrySet().stream()
            .max(porIngresos.thenComparing(e -> -e.getKey())) // empate: hora más temprana
            .orElseThrow();
    long totalDia = ingresosPorHora.values().stream().mapToLong(Long::longValue).sum();
    double ocupacion = (double) pico.getValue() / capacidad;
    return new EstadoRuta(ruta, totalDia, pico.getKey(), pico.getValue(), capacidad,
            ocupacion, ocupacion > umbral);
}

// En Main: la capacidad se entrega como una lambda
private static final Function<String, Integer> CAPACIDAD =
        ruta -> CAPACIDAD_HORARIA.getOrDefault(ruta, 10_000);
```

Dos rutas quedaron marcadas como críticas, ambas a las 18:00: la alimentadora A1, con 14.192 ingresos frente a una capacidad de 12.600 pasajeros por hora (112,6 %), y la troncal T1, con 35.095 ingresos frente a 33.000 (106,3 %). Las demás rutas operan por debajo del umbral, con ocupaciones entre 83,5 % y 88,0 %. Para la Alcaldía, esto indica que el refuerzo de flota debe concentrarse en esas dos rutas durante el pico de la tarde.

Conceptos de Programación Funcional Aplicados

Funciones Puras

Una función pura cumple dos condiciones: devuelve siempre el mismo resultado para los mismos argumentos y no produce efectos secundarios observables; es decir, no modifica variables externas, no altera sus parámetros y no realiza operaciones de entrada o salida (Urma et al., 2018). Todas las funciones de `AnalizadorTransporte` cumplen ambas condiciones:

dependen solo de la lista de registros y de los parámetros que reciben, no usan variables estáticas mutables y devuelven colecciones nuevas. También son puras `Registro.desdeCsv`, `Tramo.entre`, `evaluarRuta`, las funciones de apoyo del generador (`hash`, `mezclar`, `acumulado` e `indicePonderado`) y `Main.barra`. Las únicas operaciones con efectos, la lectura del archivo CSV, la medición del tiempo con `System.nanoTime()` y la impresión en consola, se ubicaron en `LectorCsv` y `Main`. Gracias a esa pureza, cada función pudo probarse de forma aislada con datos conocidos.

Inmutabilidad

La inmutabilidad se aplicó en tres niveles. Primero, los datos se modelaron como records (`Registro`, `Tramo`, `EstadoRuta`, `EstadisticaTiempos` y `ResultadoAnalisis`), cuyos campos son finales y que no tienen métodos de modificación (Oracle, 2021b); el constructor compacto de `Registro` valida además que ningún campo sea nulo. Segundo, todas las colecciones producidas son no modificables: las listas se crean con `toList()` o `toUnmodifiableList()`, y los mapas se envuelven con `Collections.unmodifiableMap` o `unmodifiableSortedMap` mediante `collectingAndThen`; `EstadisticaTiempos` hace además una copia defensiva del mapa que recibe. Tercero, ninguna operación altera la lista original: cuando se necesita otro orden, como en `tramosDeUsuario`, se crea una lista nueva con `sorted()`. Bloch (2018) recomienda minimizar la mutabilidad precisamente porque los objetos inmutables son seguros para hilos y pueden compartirse libremente, que es la condición para paralelizar sin condiciones de carrera.

```
public record Registro(String idUsuario, String ruta, String estacion, Accion accion,
                       LocalDateTime timestamp) {

    /** Constructor compacto: valida que ningún campo sea nulo. */
    public Registro {
        Objects.requireNonNull(idUsuario, "idUsuario");
        Objects.requireNonNull(ruta, "ruta");
        Objects.requireNonNull(estacion, "estacion");
        Objects.requireNonNull(accion, "accion");
        Objects.requireNonNull(timestamp, "timestamp");
    }
    ...
}
```

Funciones de Orden Superior

Una función de orden superior recibe otras funciones como parámetros o devuelve una función. En el proyecto se destacan cinco: `contarPor`, que recibe un `Predicate` y una `Function` y es la base de las tareas a y c; `detectarSobrecarga`, que recibe la función de capacidad horaria; `topN`, que construye comparadores con `Map.Entry.comparingByValue().reversed().thenComparing(...)`; `medir` y `mejorDe` en `Main`, que reciben como `Supplier` la tarea que se va a cronometrar, y la constante `HORA_DEL_DIA`, creada al componer dos funciones con `andThen`. Además, todas las operaciones de Stream utilizadas, como `filter`, `map`, `flatMap` y `collect`, son en sí mismas funciones de orden superior.

Expresiones Lambda y Referencias a Métodos

Las lambdas permiten pasar comportamiento como argumento de forma concisa. Se usaron, por ejemplo, para definir la capacidad de cada ruta (`ruta -> CAPACIDAD_HORARIA.getOrDefault(ruta, 10_000)`), convertir segundos a minutos (`seg -> seg / 60.0`), descartar las líneas vacías del CSV (`linea -> !linea.isBlank()`) y declarar cada prueba como un `BooleanSupplier`. Cuando la lambda solo invoca un método existente se prefirieron las referencias a métodos, que son más legibles: `Registro::estacion`, `Registro::esEntrada`, `Tramo::segundos`, `Optional::stream` y `Collections::unmodifiableMap`, entre otras.

Streams y Evaluación Perezosa

Cada tarea se expresa como un pipeline con una fuente (la lista de registros), operaciones intermedias (`filter`, `map`, `flatMap`, `sorted`, `distinct`, `limit`) y una operación terminal (`collect`, `count`, `max`, `toList`). El código describe qué se quiere obtener y no cómo recorrer la colección, sin bucles ni contadores mutables. Los colectores `groupingBy`, `counting`, `mapping`, `averagingLong`, `toSet` y `collectingAndThen` permiten construir agregaciones complejas de forma declarativa (Oracle, 2021a). Además, las operaciones intermedias son perezosas: en el generador, el flujo de horas candidatas termina en `distinct().limit(viajes)`, de modo que solo se calculan los valores necesarios para completar las horas de cada usuario, aunque el rango declarado sea de 64 candidatos.

Paralelización

Todas las funciones del analizador aceptan el parámetro `paralelo`. Con `parallelStream()`, el marco Fork/Join de Java divide la lista en segmentos que se procesan en varios hilos y luego combina los resultados parciales. Esto es seguro porque los registros son inmutables y las funciones no comparten estado; los colectores de la API se encargan de combinar los mapas parciales. El programa comprueba en cada ejecución que el resultado secuencial y el paralelo son idénticos, y una de las pruebas automatizadas verifica lo mismo con 20.000 usuarios. La Tabla 4 resume dónde se aplicó cada concepto.

Tabla 4

Conceptos de Programación Funcional y su Aplicación en el Proyecto

Concepto	Dónde se aplica	Ejemplo
Funciones puras	`AnalizadorTransporte`, `Tramo.entre`, `GeneradorDatos.hash`	`evaluarRuta(ruta, ingresos, capacidad, umbral)`
Inmutabilidad	Records del paquete `modelo`, `toList()`, `collectingAndThen`	`Collections::unmodifiableMap`
Orden superior	`contarPor`, `detectarSobrecarga`, `topN`, `medir`	`contarPor(registros, ES_ENTRADA, Registro::estacion, p)`
Lambdas y referencias	Capacidad por ruta, conversión de unidades, pruebas	`ruta -> CAPACIDAD_HORARIA.getOrDefault(ruta, 10_000)`
Streams	Tareas a–f y generador de datos	`filter → groupingBy → counting`
Paralelismo	Parámetro `paralelo` en todas las funciones	`registros.parallelStream()`

Nota. Elaboración propia.

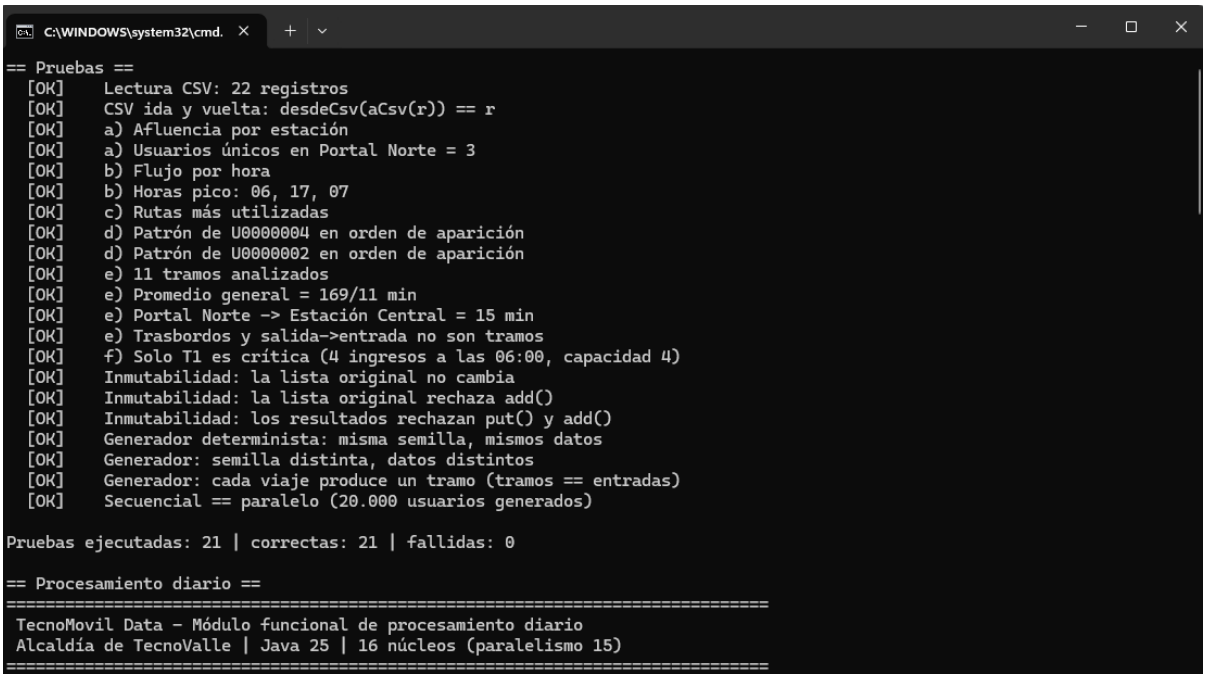
Pruebas y Evidencia de Ejecución

Pruebas Automatizadas

Como el enunciado excluye los frameworks externos, no se utilizó JUnit. La clase `PruebasAnalizador` define cada prueba como un record con un nombre y un `BooleanSupplier`, las ejecuta con un Stream y cuenta las fallidas. Los valores esperados se calcularon a mano sobre el archivo `datos/muestra_registros.csv`, que contiene 22 registros de cinco usuarios y 11 viajes diseñados para cubrir casos especiales: un trasbordo en la misma estación, una salida seguida de una entrada en otra estación, empates en el ranking de rutas y una ruta que alcanza exactamente su capacidad. También se comprueban la inmutabilidad, el determinismo del generador y la igualdad entre la ejecución secuencial y la paralela. Las 21 pruebas resultaron correctas (Figura 1).

Figura 1

Compilación y Ejecución de las Pruebas Automatizadas



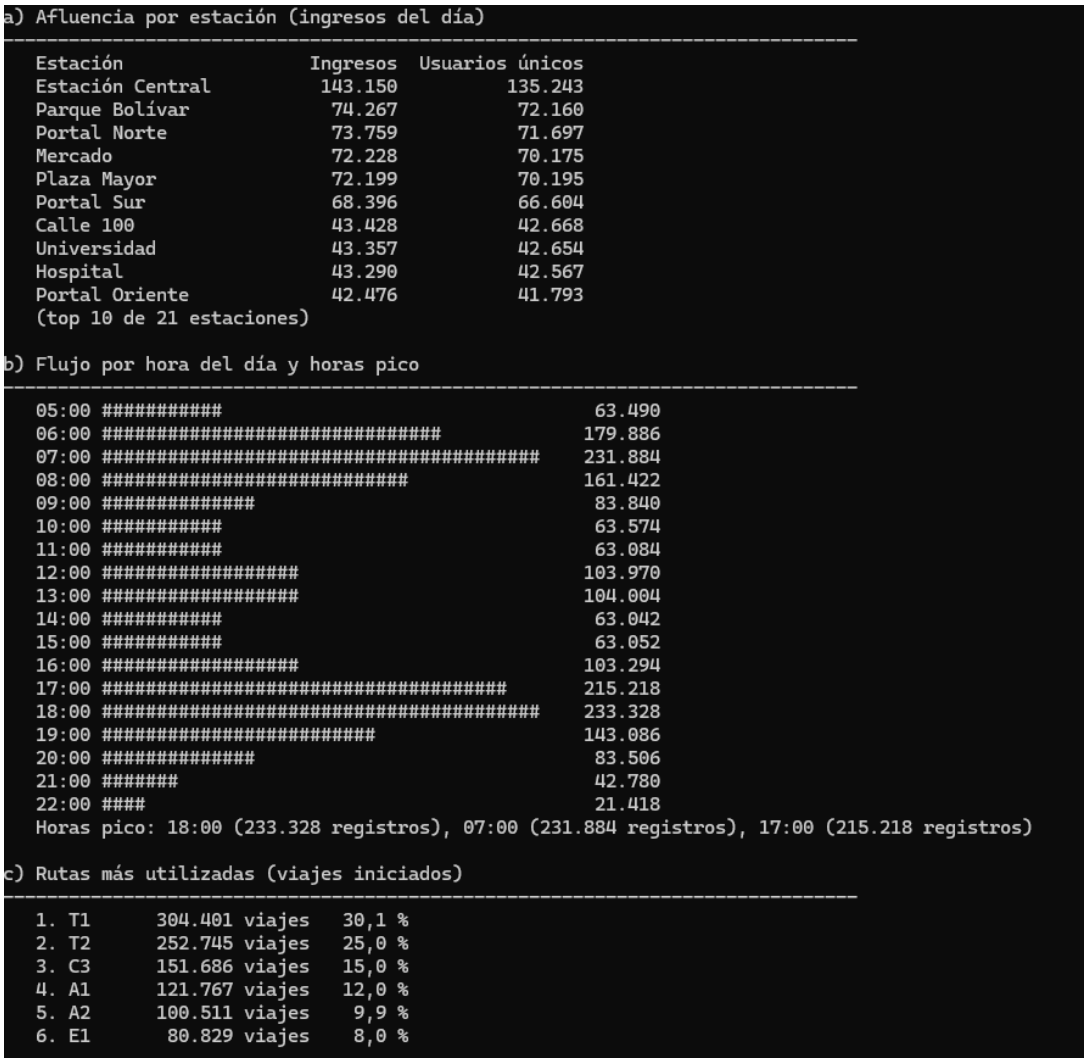
Nota. Captura de la consola de Windows (cmd) al ejecutar `compilar_y_ejecutar.bat`. Elaboración propia.

Ejecución del Procesamiento Diario

Las Figuras 2 y 3 muestran la ejecución completa del módulo sobre los 2.023.878 registros simulados de 450.000 usuarios, y la Tabla 5 resume los resultados de cada tarea.

Figura 2

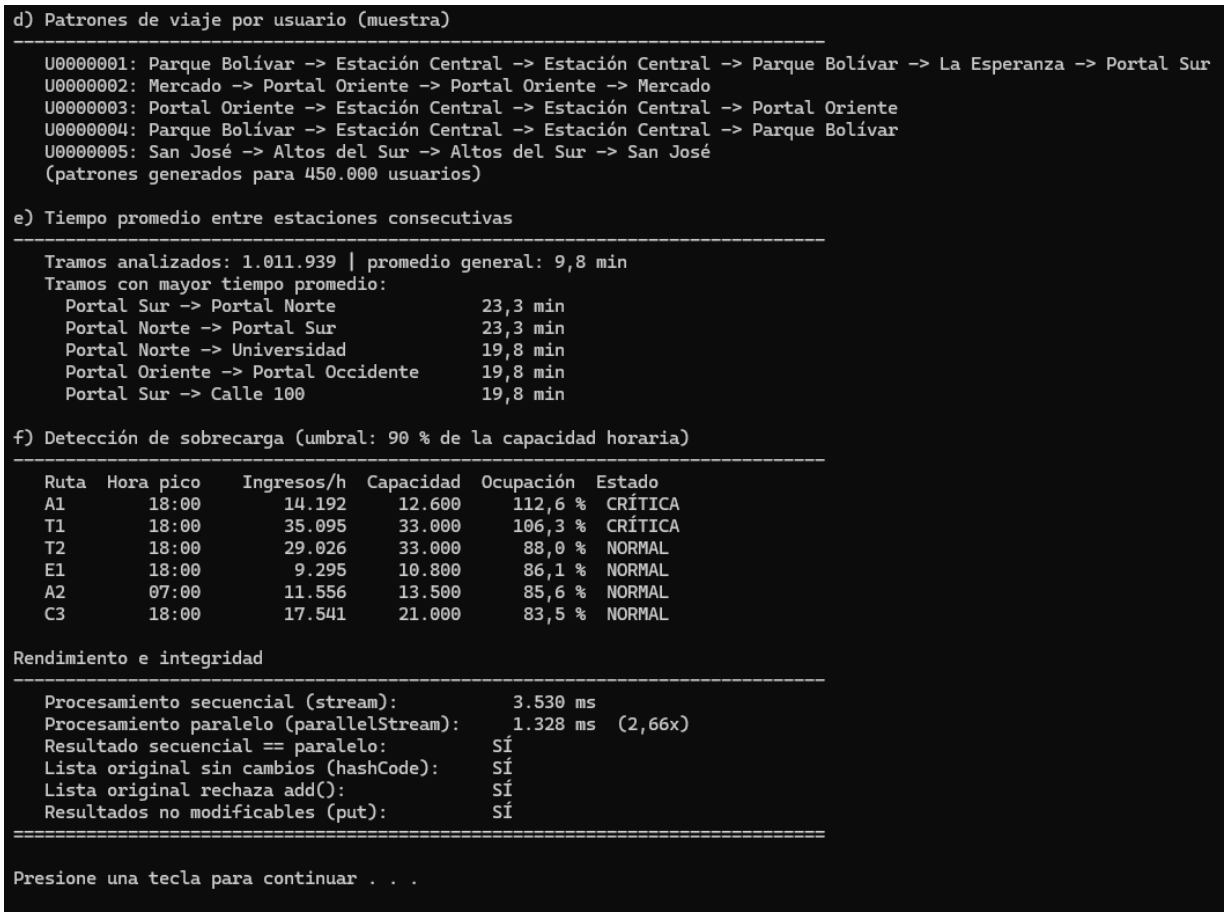
Resultados de las Tareas a, b y c



Nota. Salida en la consola de Windows (cmd) del procesamiento con 450.000 usuarios simulados, en un equipo de 16 núcleos con Java 25. Elaboración propia.

Figura 3

Resultados de las Tareas d, e y f, Rendimiento e Integridad



Nota. Continuación de la salida mostrada en la Figura 2. Elaboración propia.

Tabla 5

Resumen de Resultados del Procesamiento Diario

Tarea	Resultado principal
a) Afluencia	Estación Central: 143.150 ingresos de 135.243 usuarios distintos
b) Horas pico	18:00 (233.328 registros), 07:00 (231.884) y 17:00 (215.218)
c) Rutas	T1: 304.401 viajes (30,1 %); T2: 252.745 (25,0 %); C3: 151.686 (15,0 %)
d) Patrones	450.000 usuarios con su secuencia de estaciones visitadas
e) Tiempos	1.011.939 tramos; promedio general de 9,8 minutos
f) Sobrecarga	Rutas críticas a las 18:00: A1 (112,6 %) y T1 (106,3 %)

Nota. Datos simulados con semilla 2026 para el 30 de octubre de 2026. Elaboración propia.

Rendimiento Secuencial Frente a Paralelo

Después de una ejecución de calentamiento de la máquina virtual, cada modo se ejecutó tres veces y se tomó el mejor tiempo. La ejecución se realizó en un equipo de 16 núcleos con Java 25. Las seis tareas tardaron 3.530 ms con `stream()` y 1.328 ms con `parallelStream()`, es decir, el procesamiento paralelo fue 2,66 veces más rápido. La aceleración no llega a 16 porque una parte del trabajo no se reparte entre los hilos, como combinar los mapas parciales que produce cada hilo y ordenar los rankings finales. Más importante que la velocidad es que ambos modos produjeron exactamente el mismo resultado, que la lista original conservó su `hashCode` y que tanto la lista como los resultados rechazaron cualquier intento de modificación con una `UnsupportedOperationException` (Figura 3). Como referencia, el sistema imperativo actual tarda hasta dos horas en procesar un día (Prada Bohórquez, 2026); aunque los entornos no son directamente comparables, el módulo funcional procesa un día simulado de más de dos millones de registros en pocos segundos.

Repositorio en GitHub

La solución se versionó con Git y se alojó en la rama principal (`main`) del repositorio público **https://github.com/PabloR95/TecnoMovilData**. El repositorio contiene el código fuente, las pruebas, el archivo de datos de muestra, los scripts de compilación para Windows y Linux, el archivo `README.md` con las instrucciones de ejecución y, en la carpeta `docs`, este informe en PDF, como lo solicita la actividad. El archivo `.gitignore` excluye las clases compiladas. El proyecto se publicó con los siguientes comandos (Chacon & Straub, 2014):

```
git init -b main
git add .
git commit -m "EA2: módulo funcional TecnoMovil Data"
git remote add origin https://github.com/PabloR95/TecnoMovilData.git
git push -u origin main
```

Archivo ZIP del Proyecto

El proyecto final se entrega también comprimido en el archivo `EA2_TecnoMovilData_RamírezPablo.zip`, con la misma estructura del repositorio (Tabla 2). Para ejecutarlo basta con descomprimirlo y ejecutar `compilar_y_ejecutar.bat` en Windows o `compilar_y_ejecutar.sh` en Linux y macOS; el único requisito es tener instalado el JDK 17 o una versión superior. El programa acepta como argumento el número de usuarios que se van a simular (por ejemplo, `1000000`) o la opción `--csv` seguida de la ruta de un archivo de registros.

Conclusiones

Las seis tareas del caso de estudio se resolvieron con funciones puras y pipelines de Streams, sin bucles explícitos ni variables mutables en el núcleo del procesamiento. La función de orden superior `contarPor` mostró que pasar comportamiento como parámetro elimina el código repetido: la afluencia por estación y el uso por ruta, dos requerimientos distintos, se resolvieron con la misma función cambiando solo la clave de agrupación.

El enfoque declarativo permitió procesar 2.023.878 registros en pocos segundos sin modificar la lista original. La inmutabilidad, garantizada por los records y por las colecciones no modificables, hace que cualquier intento de alterar los datos produzca una excepción inmediata, en lugar de un error silencioso en los conteos como los que hoy afectan al sistema de TecnoValle.

La ausencia de efectos secundarios es lo que hace segura la paralelización: el resultado de `parallelStream()` fue idéntico al de la ejecución secuencial, con una aceleración de 2,66 veces en un equipo de 16 núcleos. Este diseño ataca directamente las condiciones de carrera del sistema actual. Además, al ser puras, las funciones se verificaron de forma aislada con 21 pruebas automatizadas, todas exitosas, sin necesidad de frameworks externos.

El repositorio en GitHub, con la solución en la rama `main`, un README con instrucciones y el informe del trabajo, deja el módulo listo para integrarse al sistema TecnoMovil Data y para que otros desarrolladores lo revisen, lo ejecuten y lo amplíen. Como aprendizaje general, separar un núcleo funcional de una capa imperativa delgada resultó una forma práctica de aplicar la programación funcional en Java: el procesamiento se vuelve predecible y fácil de probar, y los efectos quedan concentrados en pocos puntos del programa.

Enlaces del Repositorio

La actividad se desarrolló de forma individual. El enlace del repositorio del único integrante se presenta en la Tabla 6.

Tabla 6

Repositorio del Integrante

Integrante	**Usuario de GitHub**	**Repositorio (rama main)**
Pablo José Ramírez Acevedo	PabloR95	https://github.com/PabloR95/TecnoMovilData

Nota. Elaboración propia.

Referencias

Bloch, J. (2018). *Effective Java* (3.ª ed.). Addison-Wesley.

Chacon, S., & Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. https://git-scm.com/book/es/v2

Oracle. (2021a). *Package java.util.stream* [Documentación de la API de Java SE 17]. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/stream/package-summary.html

Oracle. (2021b). *Record classes* [Guía del lenguaje Java SE 17]. https://docs.oracle.com/en/java/javase/17/language/records.html

Prada Bohórquez, O. P. (2026). *Caso de estudio: procesamiento funcional de datos para un sistema de gestión de transporte urbano* [Material de clase, Unidad 2]. Institución Universitaria Digital de Antioquia.

Steele, G. L., Jr., Lea, D., & Flood, C. H. (2014). Fast splittable pseudorandom number generators. En *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications* (pp. 453–472). ACM. https://doi.org/10.1145/2660193.2660195

Urma, R.-G., Fusco, M., & Mycroft, A. (2018). *Modern Java in action: Lambdas, streams, functional and reactive programming*. Manning.

Apéndice A

Código Fuente del Proyecto

A continuación se transcribe el código fuente completo del proyecto, el mismo que se encuentra en el repositorio y en el archivo ZIP. Se presenta en el orden de los paquetes descritos en la Tabla 2. También se incluye el archivo de datos de muestra usado por las pruebas.

`src/tecnomovil/modelo/Accion.java`

```
package tecnomovil.modelo;

import java.util.Arrays;

/**
 * Tipo de validación registrada por los validadores y sensores del sistema.
 * ENTRADA: el usuario ingresa a la estación. SALIDA: valida para salir o cambia de ruta.
 */
public enum Accion {
    ENTRADA("entrada"),
    SALIDA("salida");

    private final String texto;

    Accion(String texto) {
        this.texto = texto;
    }

    public String texto() {
        return texto;
    }

    /** Función pura: traduce el texto del registro ("entrada" / "salida") al enum. */
    public static Accion desdeTexto(String valor) {
        return Arrays.stream(values())
                .filter(a -> a.texto.equalsIgnoreCase(valor.trim()))
                .findFirst()
                .orElseThrow(() -> new IllegalArgumentException("Acción desconocida: " + valor));
    }
}
```

`src/tecnomovil/modelo/Registro.java`

```
package tecnomovil.modelo;

import java.time.LocalDateTime;
import java.util.Objects;

/**
 * Registro inmutable de un evento del sistema de transporte de TecnoValle.
 *
 * <p>Al ser un {@code record}, todos sus campos son {@code private final}, no tiene setters
 * y Java genera equals, hashCode y toString. Una vez creado, un registro no puede cambiar,
 * por lo que puede compartirse sin riesgo entre hilos (parallelStream).</p>
 */
public record Registro(String idUsuario, String ruta, String estacion, Accion accion,
                       LocalDateTime timestamp) {

    /** Constructor compacto: valida que ningún campo sea nulo. */
    public Registro {
        Objects.requireNonNull(idUsuario, "idUsuario");
        Objects.requireNonNull(ruta, "ruta");
        Objects.requireNonNull(estacion, "estacion");
        Objects.requireNonNull(accion, "accion");
        Objects.requireNonNull(timestamp, "timestamp");
    }
```

```
    public boolean esEntrada() {
        return accion == Accion.ENTRADA;
    }

    public boolean esSalida() {
        return accion == Accion.SALIDA;
    }

    /** Función pura: convierte una línea CSV (idUsuario,ruta,estacion,accion,timestamp). */
    public static Registro desdeCsv(String linea) {
        String[] c = linea.split(",");
        return new Registro(c[0].trim(), c[1].trim(), c[2].trim(),
                Accion.desdeTexto(c[3]), LocalDateTime.parse(c[4].trim()));
    }

    /** Función pura: representación CSV del registro. */
    public String aCsv() {
        return String.join(",", idUsuario, ruta, estacion, accion.texto(), timestamp.toString());
    }
}
```

src/tecnomovil/modelo/Tramo.java

```
package tecnomovil.modelo;

import java.time.Duration;
import java.util.Optional;

/**
 * Desplazamiento de un usuario entre dos estaciones consecutivas:
 * una ENTRADA en la estación de origen seguida de una SALIDA en otra estación.
 */
public record Tramo(String origen, String destino, long segundos) {

    public String nombre() {
        return origen + " -> " + destino;
    }

    public double minutos() {
        return segundos / 60.0;
    }

    /**
     * Función pura: dados dos registros consecutivos del mismo usuario, devuelve el tramo
     * solo si forman un viaje (entrada en A y salida en B, con A distinto de B).
     * Un trasbordo (salida y entrada en la misma estación) no es un tramo.
     */
    public static Optional<Tramo> entre(Registro a, Registro b) {
        boolean esViaje = a.esEntrada() && b.esSalida() && !a.estacion().equals(b.estacion());
        return esViaje
                ? Optional.of(new Tramo(a.estacion(), b.estacion(),
                        Duration.between(a.timestamp(), b.timestamp()).toSeconds()))
                : Optional.empty();
    }
}
```

src/tecnomovil/modelo/EstadisticaTiempos.java

```
package tecnomovil.modelo;

import java.util.Collections;
import java.util.Map;
import java.util.TreeMap;

/**
 * Resultado inmutable del cálculo de tiempos entre estaciones consecutivas.
 *
 * @param totalTramos       número de tramos analizados
 * @param promedioMinutos   tiempo promedio general en minutos
 * @param promedioPorTramo  tiempo promedio (minutos) de cada tramo "Origen -> Destino"
 */
public record EstadisticaTiempos(long totalTramos, double promedioMinutos,
                                 Map<String, Double> promedioPorTramo) {

    /** Copia defensiva: el mapa recibido se copia y se expone como no modificable. */
    public EstadisticaTiempos {
        promedioPorTramo = Collections.unmodifiableSortedMap(new TreeMap<>(promedioPorTramo));
    }
}
```

src/tecnomovil/modelo/EstadoRuta.java

```
package tecnomovil.modelo;

/**
 * Estado de ocupación de una ruta durante el día.
 *
 * @param ruta              código de la ruta
 * @param ingresosDia       total de ingresos (entradas) de la ruta en el día
 * @param horaPico          hora con más ingresos en la ruta
 * @param ingresosHoraPico  ingresos registrados en esa hora pico
 * @param capacidadHoraria  capacidad simulada de pasajeros por hora de la ruta
 * @param ocupacion         ingresosHoraPico / capacidadHoraria (1.0 = 100 %)
 * @param critica           true si la ocupación supera el umbral definido
 */
public record EstadoRuta(String ruta, long ingresosDia, int horaPico, long ingresosHoraPico,
                         int capacidadHoraria, double ocupacion, boolean critica) {

    public String estado() {
        return critica ? "CRÍTICA" : "NORMAL";
    }
}
```

src/tecnomovil/datos/GeneradorDatos.java

```
package tecnomovil.datos;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.Comparator;
import java.util.List;
import java.util.stream.IntStream;
import java.util.stream.LongStream;
import java.util.stream.Stream;

import tecnomovil.modelo.Accion;
import tecnomovil.modelo.Registro;

/**
 * Generador determinista de registros simulados del sistema de transporte de TecnoValle.
 *
 * <p>No usa {@code java.util.Random} (que guarda estado mutable). Cada valor "aleatorio" se
 * obtiene con una función hash pura de (semilla, usuario, viaje, campo): la misma entrada
 * produce siempre el mismo valor. Así la generación es reproducible, no tiene efectos
 * secundarios y puede ejecutarse en paralelo sin condiciones de carrera.</p>
 */
public final class GeneradorDatos {

    private GeneradorDatos() {
    }
```

```
    /** Ruta simulada: código, peso relativo de demanda y estaciones en orden de recorrido. */
    private record Ruta(String codigo, int peso, List<String> estaciones) {
    }

    /** Trayecto de un viaje: ruta y posición de las estaciones de origen y destino. */
    private record Trayecto(Ruta ruta, int origen, int destino) {
        Trayecto invertido() {
            return new Trayecto(ruta, destino, origen);
        }

        int saltos() {
            return Math.abs(destino - origen);
        }
    }

    private static final List<Ruta> RUTAS = List.of(
            new Ruta("T1", 30, List.of("Portal Norte", "Calle 100", "Estación Central",
                    "Parque Bolívar", "Hospital", "Universidad", "Portal Sur")),
            new Ruta("T2", 25, List.of("Portal Oriente", "Mercado", "Estación Central",
                    "Plaza Mayor", "Terminal", "Portal Occidente")),
            new Ruta("C3", 15, List.of("Estación Central", "Catedral", "Parque Bolívar",
                    "Plaza Mayor", "Mercado")),
            new Ruta("A1", 12, List.of("Portal Norte", "Los Pinos", "Villa Verde", "El Bosque")),
            new Ruta("A2", 10, List.of("Portal Sur", "San José", "La Esperanza", "Altos del Sur")),
            new Ruta("E1", 8, List.of("Estación Central", "Zona Franca", "Aeropuerto")));

    /** Primera hora de operación (05:00). Los pesos cubren de 05:00 a 22:00. */
    private static final int HORA_INICIAL = 5;
    /** Peso de demanda por hora: picos en la mañana (06-07) y en la tarde (17-18). */
    private static final List<Integer> PESO_HORA =
            List.of(3, 9, 12, 8, 4, 3, 3, 5, 5, 3, 3, 5, 11, 12, 7, 4, 2, 1);
    /** Probabilidad (%) de que un usuario haga 1, 2, 3 o 4 viajes en el día. */
    private static final List<Integer> PESO_VIAJES = List.of(15, 55, 20, 10);

    private static final List<Integer> ACUM_HORA = acumulado(PESO_HORA);
    private static final List<Integer> ACUM_VIAJES = acumulado(PESO_VIAJES);
    private static final List<Integer> ACUM_RUTA =
            acumulado(RUTAS.stream().map(Ruta::peso).toList());

    /**
     * Genera los registros de un día para la cantidad de usuarios indicada.
     * La lista resultante es no modificable y está ordenada por fecha y hora,
     * como llegarían los eventos desde los validadores.
     */
    public static List<Registro> generar(int usuarios, long semilla, LocalDate fecha) {
        return IntStream.range(0, usuarios)
                .parallel()
                .boxed()
                .flatMap(u -> registrosDeUsuario(u, semilla, fecha))
                .sorted(Comparator.comparing(Registro::timestamp)
                        .thenComparing(Registro::idUsuario))
                .toList();
    }

    /** Todos los registros (entrada y salida de cada viaje) de un usuario en el día. */
    static Stream<Registro> registrosDeUsuario(int u, long semilla, LocalDate fecha) {
        String id = String.format("U%07d", u + 1);
        int viajes = 1 + indicePonderado(ACUM_VIAJES, hash(semilla, u, 0));
        // Horas distintas para cada viaje (evaluación perezosa: se generan solo las necesarias).
        List<Integer> horas = IntStream.range(0, 64)
                .map(i -> HORA_INICIAL + indicePonderado(ACUM_HORA, hash(semilla, u, 1, i)))
                .distinct()
                .limit(viajes)
                .sorted()
                .boxed()
                .toList();
        return IntStream.range(0, horas.size())
                .boxed()
                .flatMap(k -> viaje(id, u, k, horas.get(k), semilla, fecha));
    }

    /** Un viaje genera dos registros: ENTRADA en el origen y SALIDA en el destino. */
    static Stream<Registro> viaje(String id, int u, int k, int hora, long semilla,
                                  LocalDate fecha) {
        Trayecto t = trayecto(u, k, semilla);
        LocalDateTime entrada = fecha.atTime(hora,
```

```
                entero(hash(semilla, u, 5, k), 26),            // minuto 0 a 25
                entero(hash(semilla, u, 6, k), 60));           // segundo 0 a 59
        long segundosPorEstacion = 150 + entero(hash(semilla, u, 7, k), 90);  // 2,5 a 4 min
        long espera = 60 + entero(hash(semilla, u, 8, k), 120);               // 1 a 3 min
        long duracion = Math.round((t.saltos() * segundosPorEstacion + espera)
                * factorCongestion(hora));
        List<String> estaciones = t.ruta().estaciones();
        String ruta = t.ruta().codigo();
        return Stream.of(
                new Registro(id, ruta, estaciones.get(t.origen()), Accion.ENTRADA, entrada),
                new Registro(id, ruta, estaciones.get(t.destino()), Accion.SALIDA,
                        entrada.plusSeconds(duracion)));
    }

    /** El segundo viaje del día es el regreso del primero (patrón casa-trabajo-casa). */
    static Trayecto trayecto(int u, int k, long semilla) {
        if (k == 1) {
            return trayecto(u, 0, semilla).invertido();
        }
        Ruta ruta = RUTAS.get(indicePonderado(ACUM_RUTA, hash(semilla, u, 2, k)));
        int n = ruta.estaciones().size();
        int origen = entero(hash(semilla, u, 3, k), n);
        int destino = (origen + 1 + entero(hash(semilla, u, 4, k), n - 1)) % n;
        return new Trayecto(ruta, origen, destino);
    }

    /** En horas pico los viajes tardan un 20 % más. */
    static double factorCongestion(int hora) {
        return (hora == 6 || hora == 7 || hora == 17 || hora == 18) ? 1.2 : 1.0;
    }

    // ------------------------------------------------------------------------------------
    // Funciones puras de apoyo
    // ------------------------------------------------------------------------------------

    /** Sumas acumuladas de una lista de pesos: [3, 9, 12] -> [3, 12, 24]. */
    static List<Integer> acumulado(List<Integer> pesos) {
        return IntStream.rangeClosed(1, pesos.size())
                .map(i -> pesos.subList(0, i).stream().mapToInt(Integer::intValue).sum())
                .boxed()
                .toList();
    }

    /** Elige un índice según los pesos acumulados (selección ponderada). */
    static int indicePonderado(List<Integer> acumulado, long hash) {
        int r = entero(hash, acumulado.get(acumulado.size() - 1));
        return IntStream.range(0, acumulado.size())
                .filter(i -> acumulado.get(i) > r)
                .findFirst()
                .orElseThrow();
    }

    /** Entero en el rango [0, limite). */
    static int entero(long hash, int limite) {
        return (int) Math.floorMod(hash, (long) limite);
    }

    /** Combina la semilla con las claves mediante reduce: misma entrada, mismo resultado. */
    static long hash(long semilla, long... claves) {
        return LongStream.of(claves)
                .reduce(mezclar(semilla), (acum, clave) -> mezclar(acum ^ mezclar(clave)));
    }

    /** Función de mezcla SplitMix64 (pura, sin estado). */
    static long mezclar(long x) {
        long z = x + 0x9E3779B97F4A7C15L;
        z = (z ^ (z >>> 30)) * 0xBF58476D1CE4E5B9L;
        z = (z ^ (z >>> 27)) * 0x94D049BB133111EBL;
        return z ^ (z >>> 31);
    }
}
```

src/tecnomovil/datos/LectorCsv.java

```
package tecnomovil.datos;

import java.io.IOException;
import java.io.UncheckedIOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;
import java.util.stream.Stream;

import tecnomovil.modelo.Registro;

/**
 * Lectura declarativa de registros desde un archivo CSV con encabezado:
 * idUsuario,ruta,estacion,accion,timestamp
 *
 * <p>La lectura del archivo es el único efecto (entrada/salida) y queda aislada aquí;
 * la conversión de cada línea es la función pura {@link Registro#desdeCsv(String)}.</p>
 */
public final class LectorCsv {

    private LectorCsv() {
    }

    public static List<Registro> leer(Path archivo) {
        try (Stream<String> lineas = Files.lines(archivo, StandardCharsets.UTF_8)) {
            return lineas.skip(1)                    // encabezado
                    .filter(linea -> !linea.isBlank())
                    .map(Registro::desdeCsv)
                    .toList();                       // lista no modificable
        } catch (IOException e) {
            throw new UncheckedIOException("No se pudo leer " + archivo, e);
        }
    }
}
```

src/tecnomovil/analisis/AnalizadorTransporte.java

```
package tecnomovil.analisis;

import static java.util.stream.Collectors.averagingLong;
import static java.util.stream.Collectors.collectingAndThen;
import static java.util.stream.Collectors.counting;
import static java.util.stream.Collectors.groupingBy;
import static java.util.stream.Collectors.mapping;
import static java.util.stream.Collectors.toSet;
import static java.util.stream.Collectors.toUnmodifiableList;

import java.time.LocalDateTime;
import java.util.Collection;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;
import java.util.LongSummaryStatistics;
import java.util.Map;
import java.util.Optional;
import java.util.Set;
import java.util.SortedMap;
import java.util.TreeMap;
import java.util.function.Function;
import java.util.function.Predicate;
import java.util.stream.IntStream;
import java.util.stream.Stream;

import tecnomovil.modelo.EstadisticaTiempos;
import tecnomovil.modelo.EstadoRuta;
import tecnomovil.modelo.Registro;
import tecnomovil.modelo.Tramo;

/**
 * Núcleo funcional de TecnoMovil Data.
 *
 * <p>Todas las operaciones son <b>funciones puras</b>: su resultado depende solo de sus
```

```
 * parámetros, no leen ni escriben estado compartido, no hacen entrada/salida y nunca
 * modifican la lista recibida. Cada una devuelve colecciones nuevas y no modificables.
 * Por eso pueden ejecutarse con {@code stream()} o con {@code parallelStream()} y producir
 * exactamente el mismo resultado (parámetro {@code paralelo}).</p>
 */
public final class AnalizadorTransporte {

    private AnalizadorTransporte() {
        // Clase de utilidades: no se instancia.
    }

    /** Composición de funciones: Registro -> LocalDateTime -> hora del día (0-23). */
    public static final Function<Registro, Integer> HORA_DEL_DIA =
            ((Function<Registro, LocalDateTime>) Registro::timestamp)
                    .andThen(LocalDateTime::getHour);

    /** Predicado reutilizable: el registro corresponde a un ingreso a la estación. */
    public static final Predicate<Registro> ES_ENTRADA = Registro::esEntrada;

    // ---------------------------------------------------------------------------------
    // Utilidades de orden superior
    // ---------------------------------------------------------------------------------

    /** Crea el flujo secuencial o paralelo sobre cualquier colección, sin modificarla. */
    static <T> Stream<T> flujo(Collection<T> datos, boolean paralelo) {
        return paralelo ? datos.parallelStream() : datos.stream();
    }

    /**
     * Función de orden superior: recibe un predicado (qué registros contar) y una función
     * clasificadora (por qué clave agrupar) y devuelve el conteo por clave.
     */
    public static <K> Map<K, Long> contarPor(List<Registro> registros,
                                             Predicate<? super Registro> filtro,
                                             Function<? super Registro, ? extends K> clave,
                                             boolean paralelo) {
        return flujo(registros, paralelo)
                .filter(filtro)
                .collect(collectingAndThen(groupingBy(clave, counting()),
                        Collections::unmodifiableMap));
    }

    /** Ranking descendente (top N); en empate ordena por clave para ser determinista. */
    public static <K extends Comparable<? super K>, V extends Comparable<? super V>>
            List<Map.Entry<K, V>> topN(Map<K, V> valores, int n) {
        return valores.entrySet().stream()
                .sorted(Map.Entry.<K, V>comparingByValue().reversed()
                        .thenComparing(Map.Entry.<K, V>comparingByKey()))
                .limit(n)
                .map(e -> Map.entry(e.getKey(), e.getValue()))
                .toList();
    }

    // ---------------------------------------------------------------------------------
    // a) Afluencia por estación
    // ---------------------------------------------------------------------------------

    /** Cantidad de ingresos (registros de ENTRADA) por estación durante el día. */
    public static Map<String, Long> afluenciaPorEstacion(List<Registro> registros,
                                                         boolean paralelo) {
        return contarPor(registros, ES_ENTRADA, Registro::estacion, paralelo);
    }

    /** Cantidad de usuarios distintos que ingresaron a cada estación. */
    public static Map<String, Integer> usuariosUnicosPorEstacion(List<Registro> registros,
                                                                 boolean paralelo) {
        return flujo(registros, paralelo)
                .filter(ES_ENTRADA)
                .collect(collectingAndThen(
                        groupingBy(Registro::estacion,
                                collectingAndThen(mapping(Registro::idUsuario, toSet()),
                                        Set::size)),
                        Collections::unmodifiableMap));
    }

    // ---------------------------------------------------------------------------------
```

```
    // b) Horas pico
    // ------------------------------------------------------------------------------------

    /** Flujo total (entradas + salidas) agrupado por hora del día, ordenado por hora. */
    public static SortedMap<Integer, Long> flujoPorHora(List<Registro> registros,
                                                        boolean paralelo) {
        return flujo(registros, paralelo)
                .collect(collectingAndThen(
                        groupingBy(HORA_DEL_DIA, TreeMap::new, counting()),
                        Collections::unmodifiableSortedMap));
    }

    /** Las n horas con mayor flujo de registros. */
    public static List<Map.Entry<Integer, Long>> horasPico(List<Registro> registros, int n,
                                                           boolean paralelo) {
        return topN(flujoPorHora(registros, paralelo), n);
    }

    // ------------------------------------------------------------------------------------
    // c) Rutas más utilizadas
    // ------------------------------------------------------------------------------------

    /** Volumen de uso por ruta: número de viajes iniciados (entradas) en cada ruta. */
    public static Map<String, Long> usoPorRuta(List<Registro> registros, boolean paralelo) {
        return contarPor(registros, ES_ENTRADA, Registro::ruta, paralelo);
    }

    /** Las n rutas con mayor volumen de uso. */
    public static List<Map.Entry<String, Long>> rutasMasUtilizadas(List<Registro> registros,
                                                                   int n, boolean paralelo) {
        return topN(usoPorRuta(registros, paralelo), n);
    }

    // ------------------------------------------------------------------------------------
    // d) Patrones de viaje por usuario
    // ------------------------------------------------------------------------------------

    /**
     * Para cada usuario, las estaciones visitadas en el orden en que aparecen en los registros.
     * groupingBy conserva el orden de encuentro dentro de cada grupo incluso en paralelo.
     */
    public static Map<String, List<String>> patronesDeViaje(List<Registro> registros,
                                                             boolean paralelo) {
        return flujo(registros, paralelo)
                .collect(collectingAndThen(
                        groupingBy(Registro::idUsuario, TreeMap::new,
                                mapping(Registro::estacion, toUnmodifiableList())),
                        Collections::unmodifiableSortedMap));
    }

    // ------------------------------------------------------------------------------------
    // e) Tiempo promedio entre estaciones
    // ------------------------------------------------------------------------------------

    /** Todos los tramos (entrada en A seguida de salida en B) de todos los usuarios. */
    public static List<Tramo> tramos(List<Registro> registros, boolean paralelo) {
        Map<String, List<Registro>> porUsuario = flujo(registros, paralelo)
                .collect(groupingBy(Registro::idUsuario));
        return flujo(porUsuario.values(), paralelo)
                .flatMap(AnalizadorTransporte::tramosDeUsuario)
                .toList();
    }

    /**
     * Función pura: ordena (en una lista nueva) los eventos de un usuario por fecha y hora,
     * y forma pares consecutivos (i, i+1). Solo los pares que son viaje generan un Tramo.
     */
    static Stream<Tramo> tramosDeUsuario(List<Registro> eventos) {
        List<Registro> ordenados = eventos.stream()
                .sorted(Comparator.comparing(Registro::timestamp))
                .toList();
        return IntStream.range(0, ordenados.size() - 1)
                .mapToObj(i -> Tramo.entre(ordenados.get(i), ordenados.get(i + 1)))
                .flatMap(Optional::stream);
    }
```

```
    /** Promedio general y promedio por tramo, en minutos. */
    public static EstadisticaTiempos tiempoEntreEstaciones(List<Registro> registros,
                                                           boolean paralelo) {
        List<Tramo> tramos = tramos(registros, paralelo);
        LongSummaryStatistics general = flujo(tramos, paralelo)
                .mapToLong(Tramo::segundos)
                .summaryStatistics();
        Map<String, Double> porTramo = flujo(tramos, paralelo)
                .collect(groupingBy(Tramo::nombre,
                        collectingAndThen(averagingLong(Tramo::segundos), seg -> seg / 60.0)));
        return new EstadisticaTiempos(general.getCount(), general.getAverage() / 60.0, porTramo);
    }

    // ------------------------------------------------------------------------------------
    // f) Detección de sobrecarga en rutas
    // ------------------------------------------------------------------------------------

    /**
     * Función de orden superior: recibe la función que entrega la capacidad horaria simulada
     * de cada ruta y marca como crítica toda ruta cuya ocupación en su hora pico supere
     * el umbral.
     *
     * @param capacidadHoraria ruta -> pasajeros por hora que la ruta puede transportar
     * @param umbral           ocupación máxima aceptada (0.90 = 90 %)
     */
    public static List<EstadoRuta> detectarSobrecarga(List<Registro> registros,
                                                      Function<String, Integer> capacidadHoraria,
                                                      double umbral,
                                                      boolean paralelo) {
        Map<String, Map<Integer, Long>> ingresosPorRutaYHora = flujo(registros, paralelo)
                .filter(ES_ENTRADA)
                .collect(groupingBy(Registro::ruta, groupingBy(HORA_DEL_DIA, counting())));
        return ingresosPorRutaYHora.entrySet().stream()
                .map(e -> evaluarRuta(e.getKey(), e.getValue(),
                        capacidadHoraria.apply(e.getKey()), umbral))
                .sorted(Comparator.comparingDouble(EstadoRuta::ocupacion).reversed()
                        .thenComparing(EstadoRuta::ruta))
                .toList();
    }

    /** Función pura: calcula la hora pico, la ocupación y el estado de una ruta. */
    static EstadoRuta evaluarRuta(String ruta, Map<Integer, Long> ingresosPorHora,
                                  int capacidad, double umbral) {
        Comparator<Map.Entry<Integer, Long>> porIngresos = Map.Entry.comparingByValue();
        Map.Entry<Integer, Long> pico = ingresosPorHora.entrySet().stream()
                .max(porIngresos.thenComparing(e -> -e.getKey())) // empate: hora más temprana
                .orElseThrow();
        long totalDia = ingresosPorHora.values().stream().mapToLong(Long::longValue).sum();
        double ocupacion = (double) pico.getValue() / capacidad;
        return new EstadoRuta(ruta, totalDia, pico.getKey(), pico.getValue(), capacidad,
                ocupacion, ocupacion > umbral);
    }

    // ------------------------------------------------------------------------------------
    // Ejecución completa del módulo
    // ------------------------------------------------------------------------------------

    /** Ejecuta las seis tareas (a-f) y agrupa los resultados en un solo valor inmutable. */
    public static ResultadoAnalisis analizar(List<Registro> registros,
                                             Function<String, Integer> capacidadHoraria,
                                             double umbral,
                                             boolean paralelo) {
        return new ResultadoAnalisis(
                afluenciaPorEstacion(registros, paralelo),
                usuariosUnicosPorEstacion(registros, paralelo),
                flujoPorHora(registros, paralelo),
                horasPico(registros, 3, paralelo),
                rutasMasUtilizadas(registros, Integer.MAX_VALUE, paralelo),
                patronesDeViaje(registros, paralelo),
                tiempoEntreEstaciones(registros, paralelo),
                detectarSobrecarga(registros, capacidadHoraria, umbral, paralelo));
    }
}
```

src/tecnomovil/analisis/ResultadoAnalisis.java

```
package tecnomovil.analisis;

import java.util.List;
import java.util.Map;
import java.util.SortedMap;

import tecnomovil.modelo.EstadisticaTiempos;
import tecnomovil.modelo.EstadoRuta;

/**
 * Resultado inmutable del procesamiento diario (tareas a-f).
 * Al ser un record, dos resultados son iguales (equals) si todos sus componentes lo son;
 * esto permite comprobar que la ejecución secuencial y la paralela coinciden.
 */
public record ResultadoAnalisis(
        Map<String, Long> afluenciaPorEstacion,
        Map<String, Integer> usuariosUnicosPorEstacion,
        SortedMap<Integer, Long> flujoPorHora,
        List<Map.Entry<Integer, Long>> horasPico,
        List<Map.Entry<String, Long>> rutasMasUtilizadas,
        Map<String, List<String>> patronesDeViaje,
        EstadisticaTiempos tiempos,
        List<EstadoRuta> sobrecarga) {
}
```

src/tecnomovil/app/Main.java

```
package tecnomovil.app;

import static java.util.stream.Collectors.joining;

import java.nio.file.Path;
import java.time.LocalDate;
import java.util.Comparator;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.concurrent.ForkJoinPool;
import java.util.function.Function;
import java.util.function.Supplier;
import java.util.stream.IntStream;

import tecnomovil.analisis.AnalizadorTransporte;
import tecnomovil.analisis.ResultadoAnalisis;
import tecnomovil.datos.GeneradorDatos;
import tecnomovil.datos.LectorCsv;
import tecnomovil.modelo.Registro;

/**
 * Punto de entrada de TecnoMovil Data.
 *
 * <p>Esta clase es la "capa imperativa": obtiene los datos, mide tiempos e imprime en consola
 * (efectos secundarios). Todo el procesamiento ocurre en {@link AnalizadorTransporte},
 * que es puramente funcional.</p>
 *
 * <p>Uso: {@code java -cp out tecnomovil.app.Main [usuarios]} o
 * {@code java -cp out tecnomovil.app.Main --csv datos/muestra_registros.csv}</p>
 */
public final class Main {

    private static final Locale ES = Locale.forLanguageTag("es-CO");
    private static final int USUARIOS_POR_DEFECTO = 450_000;   // pasajeros diarios de TecnoValle
    private static final long SEMILLA = 2026L;
    private static final LocalDate FECHA = LocalDate.of(2026, 10, 30);

    /** Umbral de ocupación a partir del cual una ruta es crítica (90 % de su capacidad). */
    private static final double UMBRAL_CRITICO = 0.90;

    /** Capacidad horaria simulada por ruta = buses por hora x pasajeros por bus. */
    private static final Map<String, Integer> CAPACIDAD_HORARIA = Map.of(
            "T1", 220 * 150,   // troncal norte-sur: 220 buses articulados/h x 150 pasajeros
            "T2", 220 * 150,   // troncal oriente-occidente
            "C3", 140 * 150,   // circular centro
```

```
            "A1", 140 * 90,    // alimentadora norte: buses padrón de 90 pasajeros
            "A2", 150 * 90,    // alimentadora sur
            "E1", 120 * 90);   // expresa aeropuerto

    /** Función que se entrega como parámetro al análisis de sobrecarga (orden superior). */
    private static final Function<String, Integer> CAPACIDAD =
            ruta -> CAPACIDAD_HORARIA.getOrDefault(ruta, 10_000);

    /** Resultado de una tarea junto con su tiempo de ejecución. */
    private record Medicion<T>(T valor, long ms) {
    }

    public static void main(String[] args) {
        encabezado();

        Medicion<List<Registro>> carga = medir(() -> cargarDatos(args));
        List<Registro> registros = carga.valor();
        long entradas = registros.stream().filter(Registro::esEntrada).count();
        System.out.printf(ES, "Registros cargados: %,d (entradas: %,d | salidas: %,d) en %,d ms%n",
                registros.size(), entradas, registros.size() - entradas, carga.ms());

        int hashAntes = registros.hashCode();

        // Calentamiento de la JVM (JIT) y medición: mejor de 3 ejecuciones de cada modo.
        Supplier<ResultadoAnalisis> secuencial =
                () -> AnalizadorTransporte.analizar(registros, CAPACIDAD, UMBRAL_CRITICO, false);
        Supplier<ResultadoAnalisis> paralelo =
                () -> AnalizadorTransporte.analizar(registros, CAPACIDAD, UMBRAL_CRITICO, true);
        secuencial.get();
        paralelo.get();
        Medicion<ResultadoAnalisis> sec = mejorDe(3, secuencial);
        Medicion<ResultadoAnalisis> par = mejorDe(3, paralelo);

        imprimirReporte(sec.valor());
        imprimirVerificaciones(registros, hashAntes, sec, par);
    }

    // ------------------------------------------------------------------------------------
    // Carga de datos y medición
    // ------------------------------------------------------------------------------------

    private static List<Registro> cargarDatos(String[] args) {
        if (args.length >= 2 && args[0].equals("--csv")) {
            System.out.println("Leyendo registros desde " + args[1] + " ...");
            return LectorCsv.leer(Path.of(args[1]));
        }
        int usuarios = args.length >= 1
                ? Integer.parseInt(args[0].replace("_", ""))
                : USUARIOS_POR_DEFECTO;
        System.out.printf(ES, "Generando datos simulados: %,d usuarios, semilla %d, fecha %s ...%n",
                usuarios, SEMILLA, FECHA);
        return GeneradorDatos.generar(usuarios, SEMILLA, FECHA);
    }

    /** Función de orden superior: ejecuta la tarea recibida y mide su duración. */
    private static <T> Medicion<T> medir(Supplier<T> tarea) {
        long inicio = System.nanoTime();
        T valor = tarea.get();
        return new Medicion<>(valor, (System.nanoTime() - inicio) / 1_000_000);
    }

    private static <T> Medicion<T> mejorDe(int veces, Supplier<T> tarea) {
        return IntStream.range(0, veces)
                .mapToObj(i -> medir(tarea))
                .min(Comparator.comparingLong(Medicion::ms))
                .orElseThrow();
    }

    // ------------------------------------------------------------------------------------
    // Presentación (efectos secundarios: impresión en consola)
    // ------------------------------------------------------------------------------------

    private static void encabezado() {
        linea('=');
        System.out.println(" TecnoMovil Data - Módulo funcional de procesamiento diario");
        System.out.printf(ES, " Alcaldía de TecnoValle | Java %s | %d núcleos (paralelismo %d)%n",
```

```
                Runtime.version().feature(), Runtime.getRuntime().availableProcessors(),
                ForkJoinPool.commonPool().getParallelism());
        linea('=');
    }

    private static void imprimirReporte(ResultadoAnalisis r) {
        seccion("a) Afluencia por estación (ingresos del día)");
        System.out.printf(ES, "   %-20s %12s %16s%n", "Estación", "Ingresos", "Usuarios únicos");
        AnalizadorTransporte.topN(r.afluenciaPorEstacion(), 10).forEach(e ->
                System.out.printf(ES, "   %-20s %,12d %,16d%n",
                        e.getKey(), e.getValue(), r.usuariosUnicosPorEstacion().get(e.getKey())));
        System.out.printf(ES, "   (top 10 de %d estaciones)%n", r.afluenciaPorEstacion().size());

        seccion("b) Flujo por hora del día y horas pico");
        long maximo = r.flujoPorHora().values().stream().mapToLong(Long::longValue).max().orElse(1);
        r.flujoPorHora().forEach((hora, total) ->
                System.out.printf(ES, "   %02d:00 %-40s %,10d%n",
                        hora, barra(total, maximo, 40), total));
        System.out.println("   Horas pico: " + r.horasPico().stream()
                .map(e -> String.format(ES, "%02d:00 (%,d registros)", e.getKey(), e.getValue()))
                .collect(joining(", ")));

        seccion("c) Rutas más utilizadas (viajes iniciados)");
        long totalViajes = r.rutasMasUtilizadas().stream().mapToLong(Map.Entry::getValue).sum();
        IntStream.range(0, r.rutasMasUtilizadas().size()).forEach(i -> {
            Map.Entry<String, Long> e = r.rutasMasUtilizadas().get(i);
            System.out.printf(ES, "   %d. %-4s %,10d viajes  %5.1f %%%n",
                    i + 1, e.getKey(), e.getValue(), 100.0 * e.getValue() / totalViajes);
        });

        seccion("d) Patrones de viaje por usuario (muestra)");
        r.patronesDeViaje().entrySet().stream().limit(5).forEach(e ->
                System.out.printf("   %s: %s%n", e.getKey(), String.join(" -> ", e.getValue())));
        System.out.printf(ES, "   (patrones generados para %,d usuarios)%n",
                r.patronesDeViaje().size());

        seccion("e) Tiempo promedio entre estaciones consecutivas");
        System.out.printf(ES, "   Tramos analizados: %,d | promedio general: %.1f min%n",
                r.tiempos().totalTramos(), r.tiempos().promedioMinutos());
        System.out.println("   Tramos con mayor tiempo promedio:");
        AnalizadorTransporte.topN(r.tiempos().promedioPorTramo(), 5).forEach(e ->
                System.out.printf(ES, "     %-38s %5.1f min%n", e.getKey(), e.getValue()));

        seccion(String.format(ES,
                "f) Detección de sobrecarga (umbral: %.0f %% de la capacidad horaria)",
                UMBRAL_CRITICO * 100));
        System.out.printf(ES, "   %-5s %9s %13s %10s %10s  %s%n",
                "Ruta", "Hora pico", "Ingresos/h", "Capacidad", "Ocupación", "Estado");
        r.sobrecarga().forEach(s -> System.out.printf(ES,
                "   %-5s     %02d:00 %,13d %,10d %9.1f %%  %s%n",
                s.ruta(), s.horaPico(), s.ingresosHoraPico(), s.capacidadHoraria(),
                s.ocupacion() * 100, s.estado()));
    }

    private static void imprimirVerificaciones(List<Registro> registros, int hashAntes,
                                               Medicion<ResultadoAnalisis> sec,
                                               Medicion<ResultadoAnalisis> par) {
        seccion("Rendimiento e integridad");
        System.out.printf(ES, "   Procesamiento secuencial (stream):         %,7d ms%n", sec.ms());
        System.out.printf(ES, "   Procesamiento paralelo (parallelStream):   %,7d ms  (%.2fx)%n",
                par.ms(), (double) sec.ms() / Math.max(1, par.ms()));
        System.out.println("   Resultado secuencial == paralelo:          "
                + siNo(sec.valor().equals(par.valor())));
        System.out.println("   Lista original sin cambios (hashCode):     "
                + siNo(hashAntes == registros.hashCode()));
        System.out.println("   Lista original rechaza add():              "
                + siNo(lanzaUnsupported(() -> registros.add(registros.get(0)))));
        System.out.println("   Resultados no modificables (put):          "
                + siNo(lanzaUnsupported(() -> sec.valor().afluenciaPorEstacion().put("X", 0L))));
        linea('=');
    }

    /** Función pura: barra proporcional de caracteres '#'. */
    static String barra(long valor, long maximo, int ancho) {
        return "#".repeat((int) Math.round((double) valor * ancho / maximo));
    }
```

```
    private static boolean lanzaUnsupported(Runnable accion) {
        try {
            accion.run();
            return false;
        } catch (UnsupportedOperationException e) {
            return true;
        }
    }

    private static String siNo(boolean valor) {
        return valor ? "SÍ" : "NO";
    }

    private static void seccion(String titulo) {
        System.out.println();
        System.out.println(titulo);
        linea('-');
    }

    private static void linea(char c) {
        System.out.println(String.valueOf(c).repeat(78));
    }
}
```

test/tecnomovil/pruebas/PruebasAnalizador.java

```
package tecnomovil.pruebas;

import static java.util.Map.entry;

import java.nio.file.Path;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.function.BooleanSupplier;
import java.util.function.Function;

import tecnomovil.analisis.AnalizadorTransporte;
import tecnomovil.analisis.ResultadoAnalisis;
import tecnomovil.datos.GeneradorDatos;
import tecnomovil.datos.LectorCsv;
import tecnomovil.modelo.EstadisticaTiempos;
import tecnomovil.modelo.EstadoRuta;
import tecnomovil.modelo.Registro;

/**
 * Pruebas sin frameworks externos. Cada prueba es un nombre y una función (BooleanSupplier)
 * que devuelve true si el resultado coincide con el valor calculado a mano sobre
 * datos/muestra_registros.csv (22 registros, 5 usuarios, 11 viajes).
 */
public final class PruebasAnalizador {

    private record Prueba(String nombre, BooleanSupplier verificacion) {
    }

    private static final Function<String, Integer> CAPACIDAD_4 = ruta -> 4;
    private static final LocalDate FECHA = LocalDate.of(2026, 10, 30);

    public static void main(String[] args) {
        List<Registro> muestra = LectorCsv.leer(Path.of("datos/muestra_registros.csv"));
        List<Registro> copia = new ArrayList<>(muestra);
        ResultadoAnalisis r = AnalizadorTransporte.analizar(muestra, CAPACIDAD_4, 0.90, false);
        EstadisticaTiempos t = r.tiempos();
        List<Registro> generados = GeneradorDatos.generar(20_000, 2026L, FECHA);

        List<Prueba> pruebas = List.of(
                new Prueba("Lectura CSV: 22 registros", () -> muestra.size() == 22),
                new Prueba("CSV ida y vuelta: desdeCsv(aCsv(r)) == r",
                        () -> muestra.stream()
                                .allMatch(x -> Registro.desdeCsv(x.aCsv()).equals(x))),

                new Prueba("a) Afluencia por estación",
                        () -> r.afluenciaPorEstacion().equals(Map.of(
```

```
                                "Portal Norte", 3L, "Estación Central", 3L, "Universidad", 1L,
                                "Hospital", 1L, "Portal Oriente", 1L, "Plaza Mayor", 1L,
                                "El Bosque", 1L))),
                new Prueba("a) Usuarios únicos en Portal Norte = 3",
                        () -> r.usuariosUnicosPorEstacion().get("Portal Norte") == 3),

                new Prueba("b) Flujo por hora", () -> r.flujoPorHora().equals(Map.of(
                        6, 10L, 7, 4L, 12, 2L, 17, 5L, 18, 1L))),
                new Prueba("b) Horas pico: 06, 17, 07", () -> r.horasPico().equals(List.of(
                        entry(6, 10L), entry(17, 5L), entry(7, 4L)))),

                new Prueba("c) Rutas más utilizadas", () -> r.rutasMasUtilizadas().equals(List.of(
                        entry("T1", 6L), entry("T2", 2L), entry("A1", 1L), entry("C3", 1L),
                        entry("E1", 1L)))),

                new Prueba("d) Patrón de U0000004 en orden de aparición",
                        () -> r.patronesDeViaje().get("U0000004").equals(List.of(
                                "Portal Norte", "Estación Central", "Estación Central",
                                "Plaza Mayor", "Plaza Mayor", "Portal Oriente"))),
                new Prueba("d) Patrón de U0000002 en orden de aparición",
                        () -> r.patronesDeViaje().get("U0000002").equals(List.of(
                                "Estación Central", "Universidad", "Hospital",
                                "Estación Central"))),

                new Prueba("e) 11 tramos analizados", () -> t.totalTramos() == 11),
                new Prueba("e) Promedio general = 169/11 min",
                        () -> Math.abs(t.promedioMinutos() - 169.0 / 11) < 1e-9),
                new Prueba("e) Portal Norte -> Estación Central = 15 min",
                        () -> t.promedioPorTramo()
                                .get("Portal Norte -> Estación Central") == 15.0),
                new Prueba("e) Trasbordos y salida->entrada no son tramos",
                        () -> !t.promedioPorTramo().containsKey("Universidad -> Hospital")
                                && !t.promedioPorTramo()
                                        .containsKey("Portal Norte -> Portal Norte")),

                new Prueba("f) Solo T1 es crítica (4 ingresos a las 06:00, capacidad 4)",
                        () -> r.sobrecarga().stream()
                                .filter(EstadoRuta::critica)
                                .map(EstadoRuta::ruta)
                                .toList().equals(List.of("T1"))
                                && r.sobrecarga().get(0).horaPico() == 6
                                && r.sobrecarga().get(0).ingresosHoraPico() == 4),

                new Prueba("Inmutabilidad: la lista original no cambia",
                        () -> muestra.equals(copia)),
                new Prueba("Inmutabilidad: la lista original rechaza add()",
                        () -> lanzaUnsupported(() -> muestra.add(muestra.get(0)))),
                new Prueba("Inmutabilidad: los resultados rechazan put() y add()",
                        () -> lanzaUnsupported(() -> r.afluenciaPorEstacion().put("X", 1L))
                                && lanzaUnsupported(
                                        () -> r.patronesDeViaje().get("U0000001").add("X"))),

                new Prueba("Generador determinista: misma semilla, mismos datos",
                        () -> GeneradorDatos.generar(2_000, 7L, FECHA)
                                .equals(GeneradorDatos.generar(2_000, 7L, FECHA))),
                new Prueba("Generador: semilla distinta, datos distintos",
                        () -> !GeneradorDatos.generar(2_000, 7L, FECHA)
                                .equals(GeneradorDatos.generar(2_000, 8L, FECHA))),
                new Prueba("Generador: cada viaje produce un tramo (tramos == entradas)",
                        () -> AnalizadorTransporte.tramos(generados, true).size()
                                == generados.stream().filter(Registro::esEntrada).count()),
                new Prueba("Secuencial == paralelo (20.000 usuarios generados)",
                        () -> AnalizadorTransporte.analizar(generados, ruta -> 1_000, 0.9, false)
                                .equals(AnalizadorTransporte.analizar(
                                        generados, ruta -> 1_000, 0.9, true))));

        long fallidas = pruebas.stream()
                .filter(p -> !ejecutar(p))
                .count();
        System.out.printf("%nPruebas ejecutadas: %d | correctas: %d | fallidas: %d%n",
                pruebas.size(), pruebas.size() - fallidas, fallidas);
        System.exit(fallidas == 0 ? 0 : 1);
    }

    private static boolean ejecutar(Prueba prueba) {
        boolean ok;
```

```
        try {
            ok = prueba.verificacion().getAsBoolean();
        } catch (RuntimeException e) {
            ok = false;
        }
        System.out.println((ok ? "  [OK]    " : "  [FALLA] ") + prueba.nombre());
        return ok;
    }

    private static boolean lanzaUnsupported(Runnable accion) {
        try {
            accion.run();
            return false;
        } catch (UnsupportedOperationException e) {
            return true;
        }
    }
}
```

datos/muestra_registros.csv

```
idUsuario,ruta,estacion,accion,timestamp
U0000005,A1,El Bosque,entrada,2026-10-30T06:05:00
U0000001,T1,Portal Norte,entrada,2026-10-30T06:10:00
U0000002,T1,Estación Central,entrada,2026-10-30T06:20:00
U0000005,A1,Portal Norte,salida,2026-10-30T06:20:00
U0000005,T1,Portal Norte,entrada,2026-10-30T06:22:00
U0000001,T1,Universidad,salida,2026-10-30T06:30:00
U0000002,T1,Universidad,salida,2026-10-30T06:32:00
U0000005,T1,Estación Central,salida,2026-10-30T06:37:00
U0000004,T1,Portal Norte,entrada,2026-10-30T06:40:00
U0000004,T1,Estación Central,salida,2026-10-30T06:55:00
U0000003,T2,Portal Oriente,entrada,2026-10-30T07:00:00
U0000003,T2,Estación Central,salida,2026-10-30T07:15:00
U0000003,E1,Estación Central,entrada,2026-10-30T07:20:00
U0000003,E1,Aeropuerto,salida,2026-10-30T07:35:00
U0000004,C3,Estación Central,entrada,2026-10-30T12:00:00
U0000004,C3,Plaza Mayor,salida,2026-10-30T12:10:00
U0000001,T1,Universidad,entrada,2026-10-30T17:05:00
U0000002,T1,Hospital,entrada,2026-10-30T17:10:00
U0000002,T1,Estación Central,salida,2026-10-30T17:22:00
U0000001,T1,Portal Norte,salida,2026-10-30T17:25:00
U0000004,T2,Plaza Mayor,entrada,2026-10-30T17:40:00
U0000004,T2,Portal Oriente,salida,2026-10-30T18:00:00
```